CSEE W4824 – Computer Architecture

Homework Assignment 1

Group members: Lucy He, Jiayi Wu

UNIs: lh3365, jw4782

Question 1

We modified the starter code to iterate through memory sizes from 2^6 to 2^{21} and measured the time it takes to copy data of each size between locations in DRAM. We used _mm_mfence() to ensure all flushes are complete before each timed trial and used _mm_lfence() around timestamp reads to serialize these instructions and prevent out-of-order executions from skewing measurements. After running the experiment a few times on a CloudLab instance, we observed that the first few measurements are significantly larger than subsequent measurements, and suspected there are one-time initialization effects causing the discrepancy. Thus, we added a prefault_touch() function to walk through the buffer and trigger page faults in advance, which reduced outlier measurements by eliminating cases with extra overhead from memory allocation. To avoid the one-time setup cost at the start of our experiment, we also added a warmup phase to stabilize the environment before collecting measurements. Finally, the modified program attached in the Appendix is used to collect data and produce the statistical graphs discussed below.

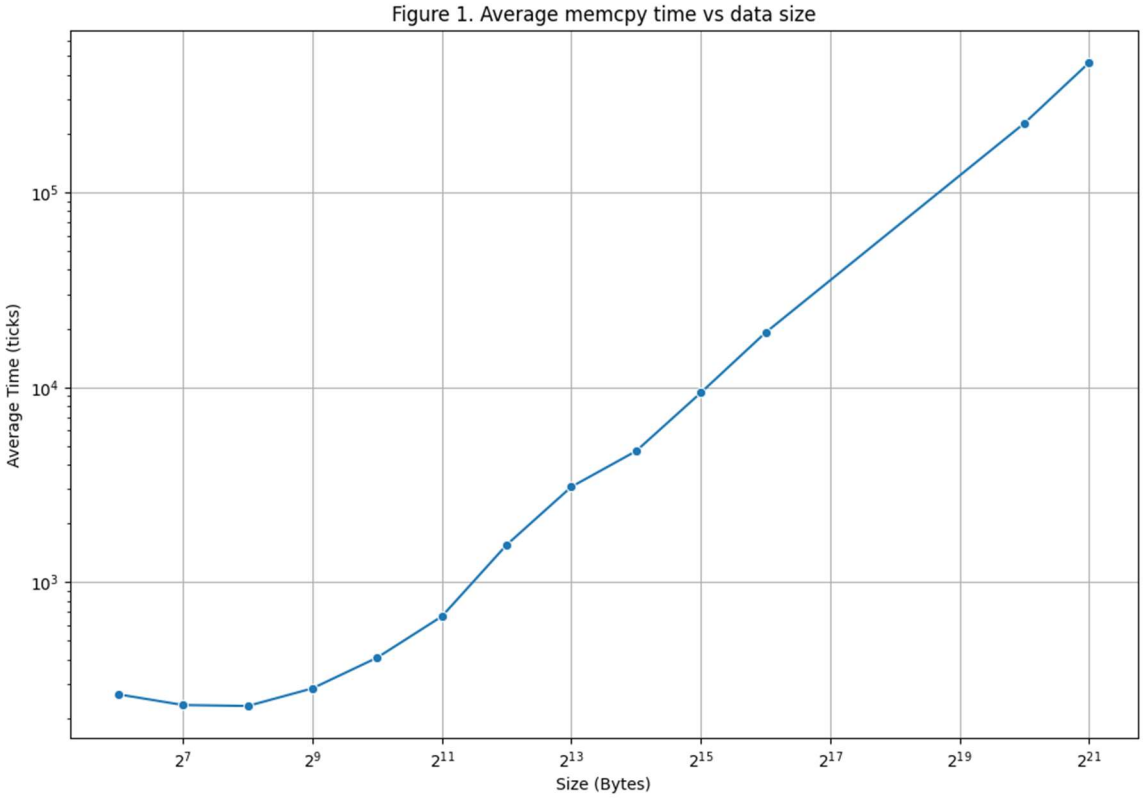
Our data is collected on a CloudLab instance of the Ubuntu24 profile, which provides a virtualized Linux environment with x86 processors and standard DDR4 DRAM modules. The L1 data, L1 instruction, L2, and L3 cache sizes are shown in the screenshot below.

```
lh3365@node1:~$ cat /sys/devices/system/cpu/cpu0/cache/index*/size
32K
32K
256K
25600K
```

Normally, the memcpy performance depends strongly on cache sizes because small data sizes can have copies in the cache, while larger data sizes spill into higher-level caches and DRAM. In this experiment, we used clflush to ensure that every memcpy goes to main memory (DRAM) rather than being served from cache. Thus, our result should be dominated by DRAM latency and bandwidth, and independent of cache boundaries.

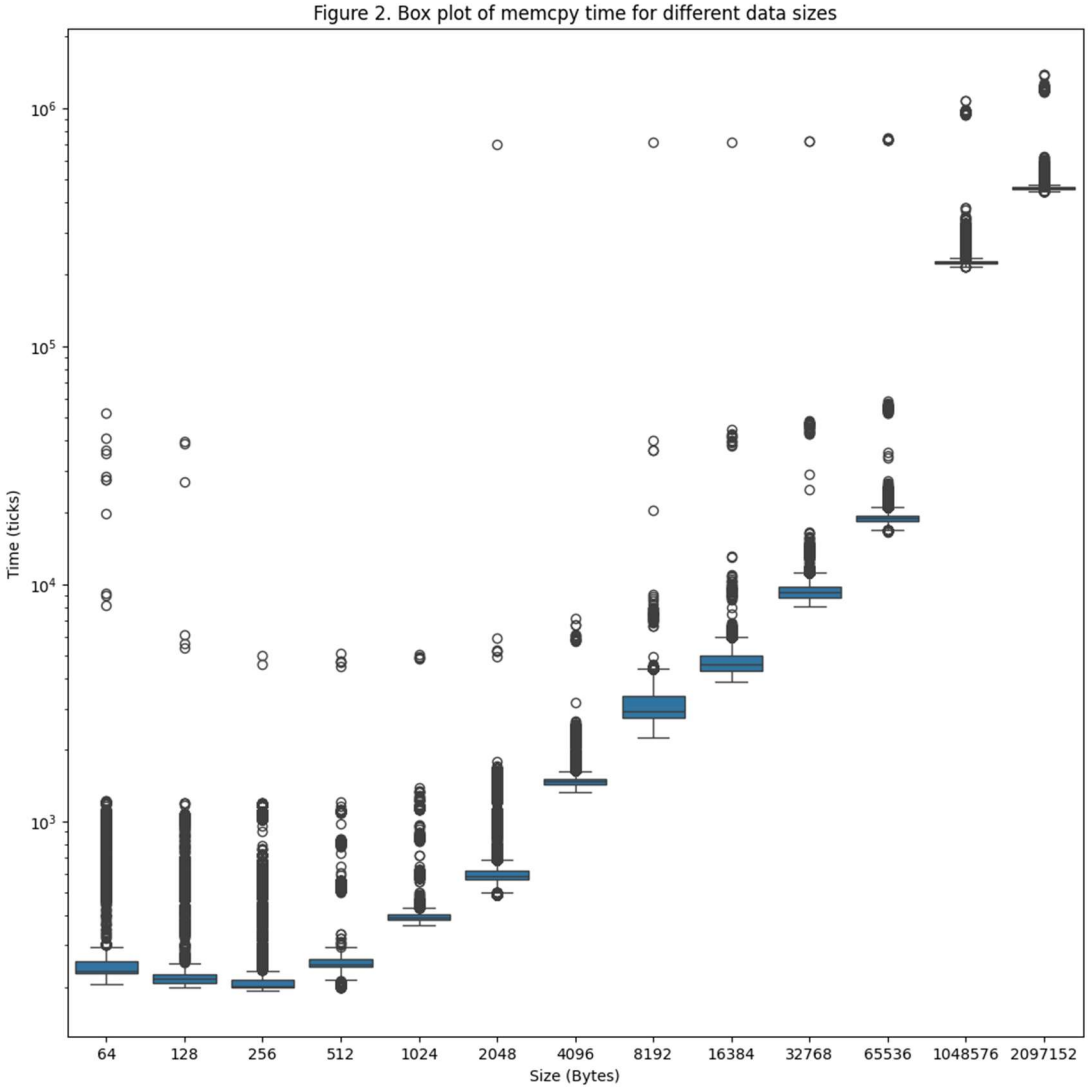
By running our code on the machine to collect data in a CSV file and using Python for data processing, we obtain an average time vs. data size plot on a log scale. In Figure 1, we observe that the copy time stays relatively constant and even slightly decreases when the data size increases from 64 to 512 Bytes. This is because the memcpy time is minimal for very small data sizes, and the measured times are dominated by fixed overheads such as function setup, memory fences, and measurement noise. As the data size increases beyond 512 Bytes, we start to see a positive trend between copy time and data size, since the variable cost of copying additional cache lines becomes dominant and the total measured time grows roughly linearly with size on a log scale.

Figure 1. Average memcpy time vs data size



Using measurements from 100,000 trials per data size, we were able to make a box plot showing the distributions of measured memcpy times in ticks for each data size (Figure 2). The box plot shows the median, interquartile range, and typical spread of the copy time for each data size, and the circles mark the outliers. From the box plot, we observed some fixed outliers across different data sizes, which suggests that there exist occasional time factors independent of the data size. For smaller data sizes between 64 and 4096 Bytes, there are fixed outliers slightly below 10^4 ticks, which are much larger than the median of these small measurements. These are likely caused by external interruptions such as OS scheduling events or timer interrupts, where the measurement includes small waiting times that are independent of the memcpy operation. For larger data sizes beyond 2048 Bytes, we also observe a few fixed outliers around 10^6 ticks, which could be over two orders of magnitude larger than normal measurements. These could be explained by system-level events such as context switches, page faults, or virtualization overhead in CloudLab, where the memcpy operation is suspended for heavier system activities, causing unusually large measurements.

Figure 2. Box plot of memcpy time for different data sizes



The time measurement distributions are plotted for each data size and are provided in the appendix. Three representative examples are presented below (Figures 3–5). In these diagrams, the extremely large outliers are truncated by removing values above the 99th percentile. Various random variabilities can explain the remaining outliers and spread. System-level events, such as context switches, interrupts, and kernel activity, can temporarily pause the execution of memcpy and add time to the measurement. TLB misses and page boundary crossings add delays by forcing extra page table lookups during address translation. The periodic DRAM refresh cycles could also introduce additional latency when memory accesses collide or wait for refresh operations to complete.

For small data sizes, such as Figure 3, the distribution is tightly clustered with a very long tail of outliers, because even short delays can take up a large fraction of the total time. At medium data sizes, such as those shown in Figure 4, the distribution becomes wider and appears multimodal because variability produces delays comparable to the normal memcpy time, creating distinct clusters of measurements. For large data sizes, such as Figure 5, the distribution is unimodal because the DRAM copy time dominates, and the variability only widens the distribution.

Figure 3. Distributions of memcpy time for size 64 Bytes

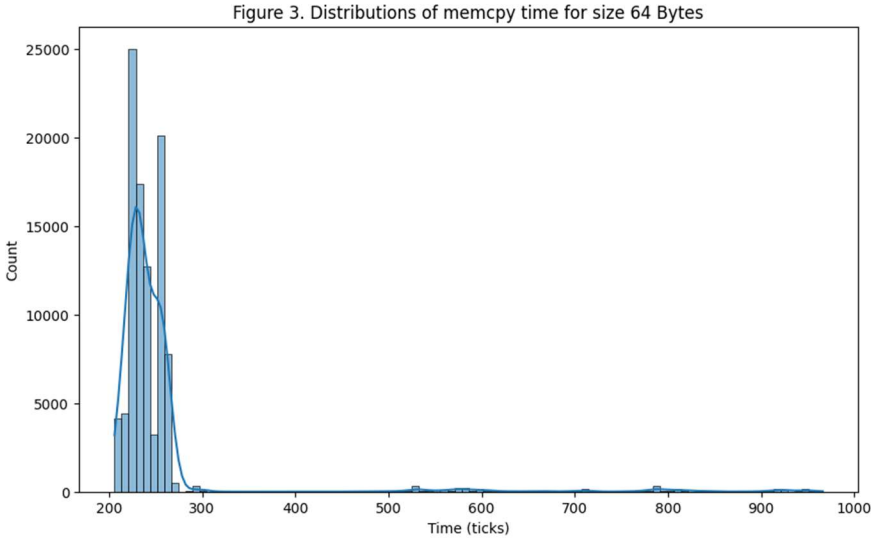


Figure 4. Distributions of memcpy time for size 16384 Bytes

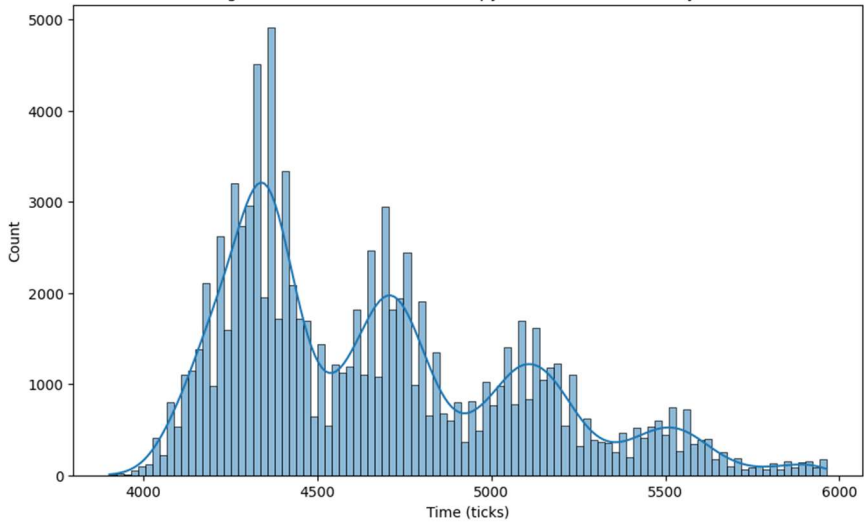
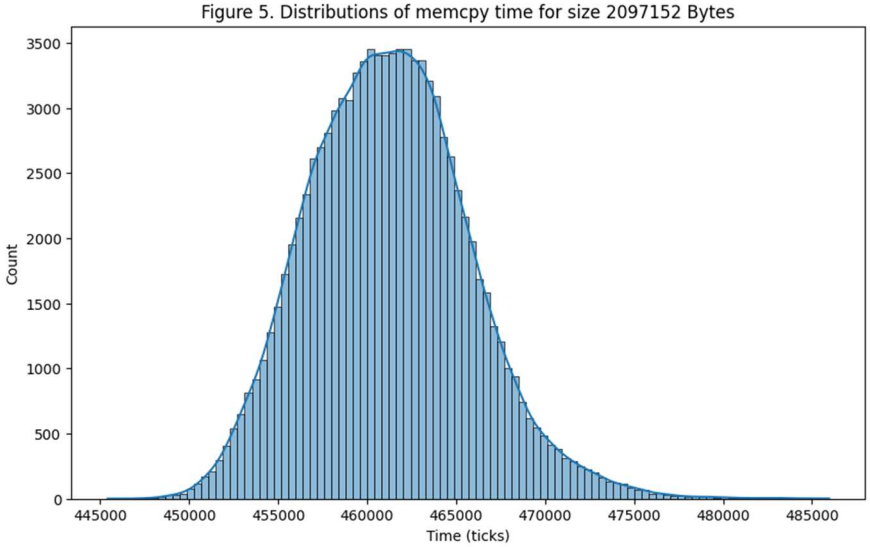


Figure 5. Distributions of memcpy time for size 2097152 Bytes



Question 2

Using the “sudo lshw -class memory” command, we were able to find the vendor and part number of the memory module on our Cloudlab instance.

```
*-memory
     description: System Memory
     physical id: 7
     slot: System board or motherboard
     size: 64GiB
     capabilities: ecc
     configuration: errordetection=multi-bit-ecc
   *-bank:0
        description: DIMM DDR4 Synchronous Registered (Buffered) 2400 MHz (0.4 ns)
        product: 809082-091
        vendor: HP
        physical id: 0
        slot: PROC 1 DIMM 1
        size: 16GiB
        width: 64 bits
        clock: 2400MHz (0.4ns)
```

Using similar commands on our personal machines and from online datasheets, we extracted the DRAM timing parameters for both our personal computers (Micron LPDDR5 SDRAM and Hynix DDR4-2666 SO-DIMM) and our CloudLab instance (HP DDR4-2400 Registered DIMMs). The results are summarized below:

Table 1. Micron LPDDR5 SDRAM

Parameter	Symbol	Typical Value
Row to Column Delay	tRCD	~18 ns
Row Precharge Time	tRP	~18 ns
CAS Latency	tCAS / tCL	~34 cycles (depending on speed bin)
Row Cycle Time	tRC	~42 ns
Row Active Time	tRAS	~32–34 ns
Row to Row Delay	tRRD	~4–6 ns (short vs long banks)
Write Recovery	tWR	~15 ns
Write to Read Delay	tWTR	~7.5–10 ns
Column Write Delay	tCWD	~12–16 cycles
Read to Precharge	tRTP	~7.5 ns
CAS-to-CAS Delay	tCCD	4 cycles (short) / 8 cycles (long)
Burst Length / Time	tBURST	BL16 = 8 cycles

Table 2. Hynix DDR4-2666 SO-DIMM

parameters	Cycles	Time (ns)
tCL/CAS latency	19	14.25
tRCD	19	14.25
tRP	19	14.25
tCK	1	0.75
tRAS	12	32
tRC	17	46.25
tRDD_S	4	3.33
tRDD_L	6	5
tWR	15	12.5
tWTR_S	4	3.33
tWTR_L	7	5.83
tCWD	16	13.33
tRTP	9	7.5
tCCD	4	3.33
tBURST	4	3.33

Table 3. HP DDR4-2400 RDIMM

parameters	Cycles	Time (ns)
tCL/CAS latency	17	14.2
tRCD	17	14.2
tRP	17	14.2
tCK	1	0.833
tRAS	39	32.5
tRC	56	46.65
tRDD_S	4	3.33
tRDD_L	6	5
tWR	15	12.5
tWTR_S	4	3.33
tWTR_L	7	5.83
tCWD	16	13.33
tRTP	9	7.5
tCCD	4	3.33
tBURST	4	3.33

In the measurement of question 1, we used the HP DDR4-2400 RDIMM on the CloudLab instance. In the worst-case scenario, the total time it takes to complete a memcpy operation is the sum of the time it takes to read the row, the constraint between read and write, and the time to write the data to the destination.

$$T = (tRP + tRCD + tCL + tBURST) + tWR + (tCWD + tBURST)$$

From the data sheet, we obtain T = (17 + 17 + 17 + 4) + 15 + (16 + 4) = 90 ticks for a single cache line. Comparing the data sheet with the results, our measured results in question 1 are much larger than the theoretical time. Some possible reasons for the discrepancies are as follows:

1) The difference between ideal and practical environments
   The datasheet values (e.g., tRCD ≈ 17 ticks, tCAS ≈ 17 ticks, tRP ≈ 17 ticks) describe individual DRAM command delays under ideal conditions.
   In contrast, the memcpy benchmark measures end-to-end memory copy latency, which includes multiple layers: CPU instruction overhead, cache hierarchy behavior, prefetching, write-back buffers, and memory controller scheduling. This aggregation makes measured times much higher than the theoretical single-parameter delays.

2) System noise and outliers
   The boxplot (Figure 2) shows many outliers, especially at larger sizes, due to OS scheduling, TLB misses, or page faults. None of these are accounted for in the raw DRAM timing table, but they significantly affect empirical memcpy performance.

Question 3

The result of the question 3 is shown below.

```
EveWu@node1:~/CSEE4824$ ./rowtest
Testing DRAM Row Buffer Policy
==============================
First access to row 1: 30 ticks
First access to row2: 40 ticks
Second access to row2: 26 ticks
EveWu@node1:~/CSEE4824$
```

According to the result, after the first access of row 1, we then accessed row 2 for the first time and observed a different-row access time of 40 ticks. After that, we repeated the access to row 2 and found that the second access time dropped to 26 ticks, which is faster than the first access. Thus, we can infer that the DRAM of CloudLab follows an open-row buffer policy, where the controller leaves a row open by keeping the data in the row buffer, thereby speeding up the second access by directly serving it from the open row.

Project logs

09/22:

Lucy He:

- Created CloudLab experiment
- Wrote a first draft of modified starter code to measure DRAM memcpy times for different data sizes and saved measurements in csv.

09/23:

Lucy He:

- Wrote python code to plot statistical distributions and box plots

Jiayi Wu:

- Enhance drafted code with mfence, warm-up phase, clflush, and disabled memcpy compiler optimization to get the result.csv of the question.

09/29:

Lucy He:

- Ran commands and searched for datasheet and timing parameters of the DIMM
- Worked on report write-up

Jiayi Wu:

- Ran commands and searched for datasheet and timing parameters of the DIMM
- Worked on report write-up
- Wrote the drafted code for the question 3

09/30:

Lucy He:

- Worked on report write-up
- Modified python code to produce clean statistical distribution plots
- Debugged code for Question 3

Jiayi Wu:

- Worked on report write-up
- Finished the code for the question 3

Appendix

Modified C code for running experiments:

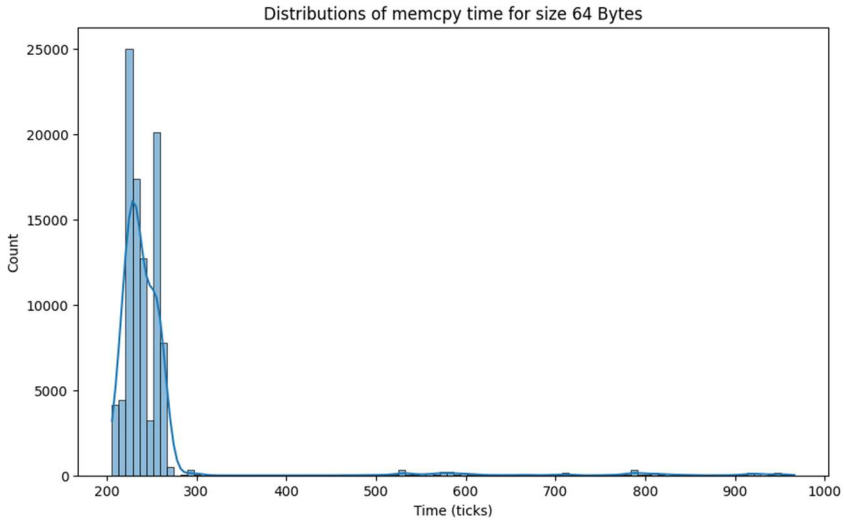
```c
#define _GNU_SOURCE
#include <stdint.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <inttypes.h>
#include <x86intrin.h>   // for _mm_clflush, __rdtscp (same as asm)

// Reduced REPEAT for quicker testing, especially for large memory sizes
#ifndef REPEAT
#define REPEAT 10000
#endif
#define WARMUP 10
#define CACHELINE 64
#define PAGE 4096

static inline void clflush_range(void *p, size_t len) {
    uintptr_t addr = (uintptr_t)p;
    uintptr_t end  = addr + len;
    for (; addr < end; addr += CACHELINE) {
        _mm_clflush((void*)addr);
    }
    _mm_mfence();  // ensure flush completion
}

static inline uint64_t tsc_begin(void) {
    unsigned int aux;
    _mm_lfence();               // serialize before reading TSC
    return __rdtscp(&aux);      // read tsc with ordering
}

static inline uint64_t tsc_end(void) {
    unsigned int aux;
    uint64_t t = __rdtscp(&aux); // read tsc with ordering
    _mm_lfence();         // serialize after reading TSC
    return t;
}

static inline void prefault_touch(char *buf, size_t bytes) {
    // Page pre-touch to avoid page faults/zeroing during timing
    for (size_t i = 0; i < bytes; i += PAGE) buf[i] = 1;
    if (bytes > 0) buf[bytes-1] ^= 0; // touch last byte to prevent OOB
}
```

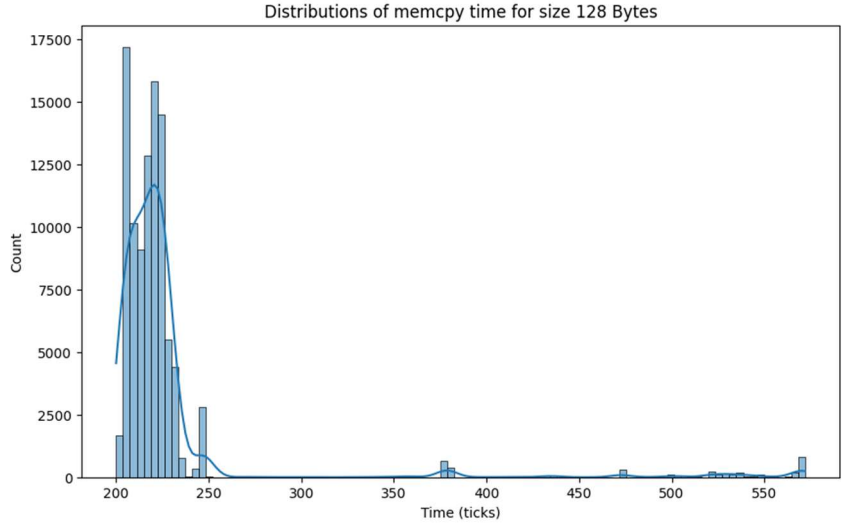
```c
static inline void memtest(size_t bytes, FILE *out) {
    // 64B aligned allocation (to avoid false sharing across cache lines)
    char *src, *dst;
    if (posix_memalign((void**)&src, CACHELINE, bytes) ||
        posix_memalign((void**)&dst, CACHELINE, bytes)) {
        perror("posix_memalign"); exit(1);
    }

    // Initialize & pre-touch pages
    memset(src, 0xA5, bytes);
    memset(dst, 0,    bytes);
    prefault_touch(src, bytes);
    prefault_touch(dst, bytes);

    // Warmup: establish i-cache path/page tables/TLBs etc., not timed
    for (int r = 0; r < WARMUP; ++r) {
        clflush_range(src, bytes);
        clflush_range(dst, bytes);
        (void)memcpy(dst, src, bytes);
    }

    // Start actual measurement
    for (int r = 0; r < REPEAT; ++r) {
        // flush all cache lines that will be touched this iteration
        clflush_range(src, bytes);
        clflush_range(dst, bytes);

        uint64_t t0 = tsc_begin();
        memcpy(dst, src, bytes);
        uint64_t t1 = tsc_end();

        // prevent compiler optimizing away memcpy
        asm volatile("" :: "r"(dst[0]) : "memory");

        // record results to CSV
        fprintf(out, "%zu,%" PRIu64 "\n", bytes, (t1 - t0));
    }

    free(src);
    free(dst);
}

int main(void) {
    FILE *out = fopen("results.csv", "w");
    if (!out) { perror("fopen"); return 1; }
    fprintf(out, "Size(Bytes),Time(Ticks)\n");

    // iterate over sizes from 2^6 to 2^21
    const int exps[] = {6,7,8,9,10,11,12,13,14,15,16,20,21};
    for (size_t i = 0; i < sizeof(exps)/sizeof(exps[0]); ++i) {
        size_t bytes = (size_t)1 << exps[i];
        memtest(bytes, out);
    }
    fclose(out);
    return 0;
}
```

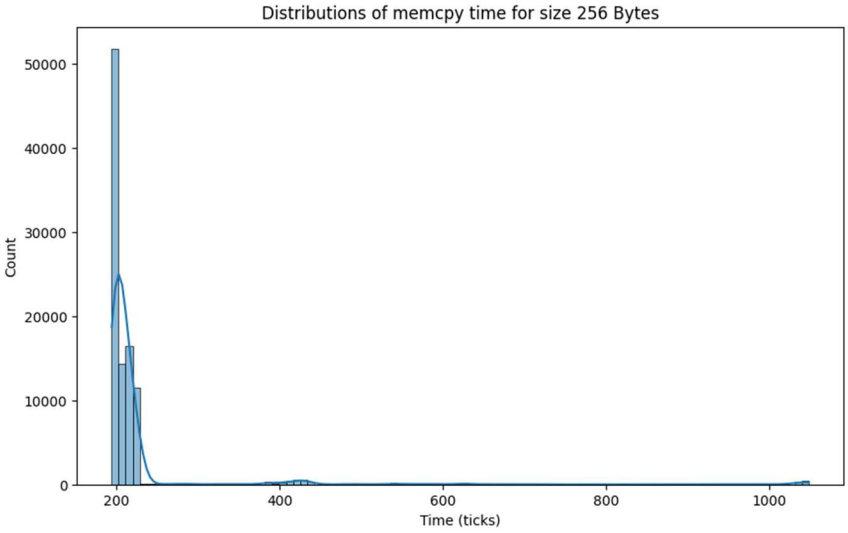
Python code to produce statistical distribution plots:

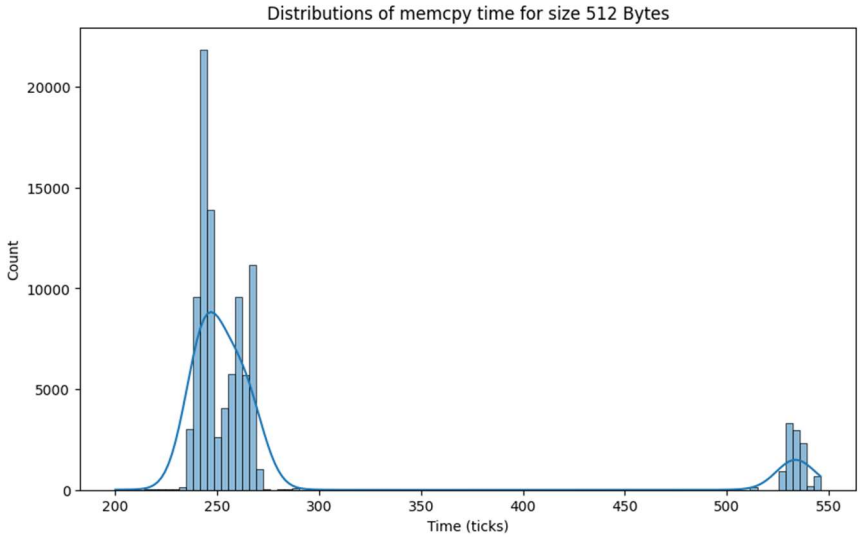
```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

df = pd.read_csv('results.csv')

# plot average time vs size
avg_times = df.groupby("Size(Bytes)")["Time(Ticks)"].mean().reset_index()
plt.figure(figsize=(12, 8))
sns.lineplot(data=avg_times, x='Size(Bytes)', y='Time(Ticks)', marker='o')
plt.xscale("log", base=2)
plt.yscale("log", base=10)
plt.title("Figure 1. Average memcpy time vs data size")
plt.xlabel("Size (Bytes)")
plt.ylabel("Average Time (ticks)")
plt.grid(True)
plt.show()

# box plot
plt.figure(figsize=(12, 12))
sns.boxplot(data=df, x='Size(Bytes)', y='Time(Ticks)')
plt.yscale("log")
plt.title("Figure 2. Box plot of memcpy time for different data sizes")
plt.xlabel("Size (Bytes)")
plt.ylabel("Time (ticks)")
plt.show()

# histograms
for size, size_df in df.groupby("Size(Bytes)"):
  # filtering outliers
  low = size_df["Time(Ticks)"].quantile(0.00)
  high = size_df["Time(Ticks)"].quantile(0.99)
  size_df = size_df[(size_df["Time(Ticks)"] >= low) & (size_df["Time(Ticks)"] <= high)]
  plt.figure(figsize=(10, 6))
  sns.histplot(data=size_df, x='Time(Ticks)', bins=100, kde=True)
  plt.title("Distributions of memcpy time for size {} Bytes".format(size))
  plt.xlabel("Time (ticks)")
  plt.ylabel("Count")
  plt.show()
```

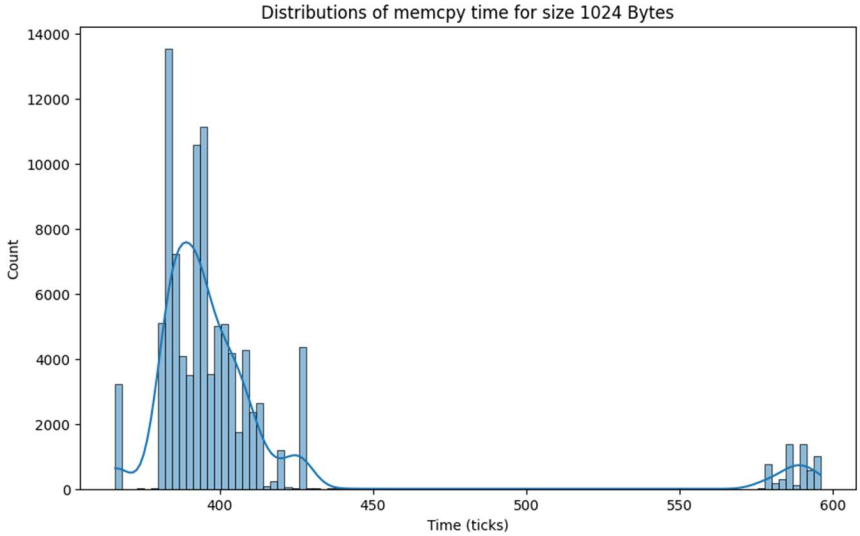
Distribution for each data size:

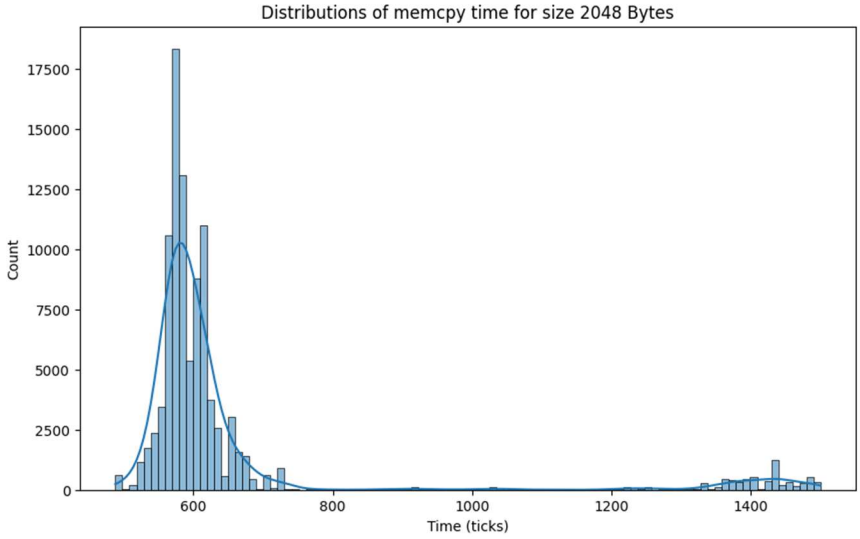


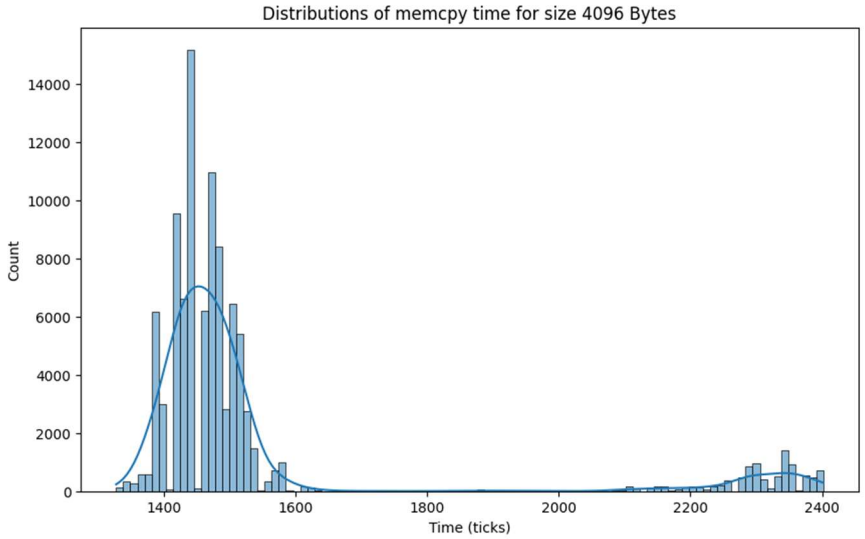


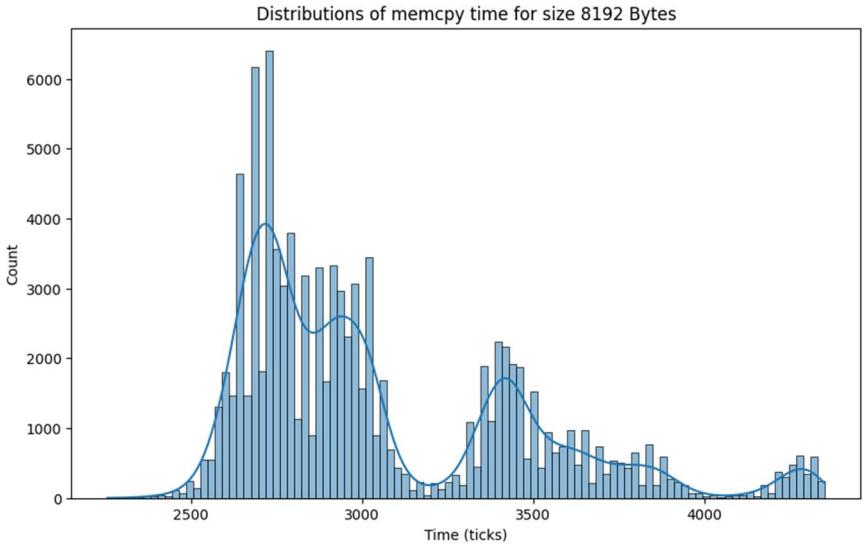


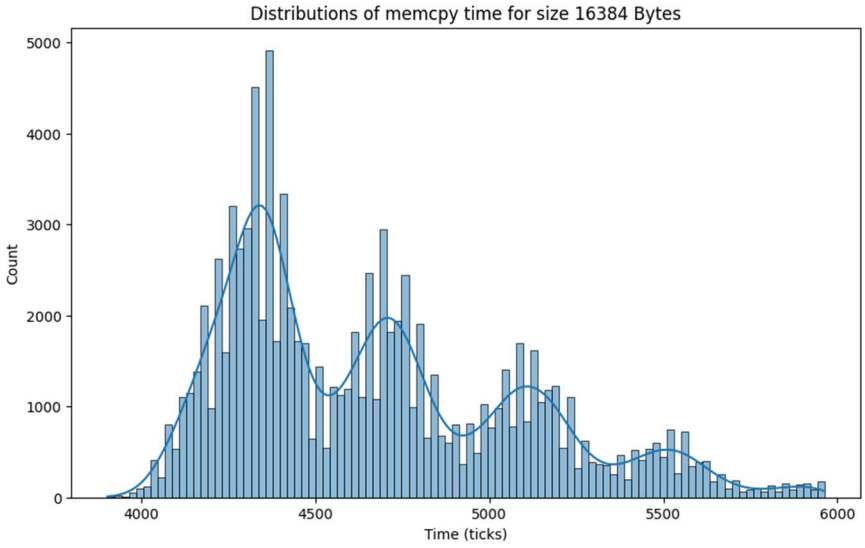

Distributions of memcpy time for size 512 Bytes
Count
Time (ticks)
0
5000
10000
15000
20000
200
250
300
350
400
450
500
550

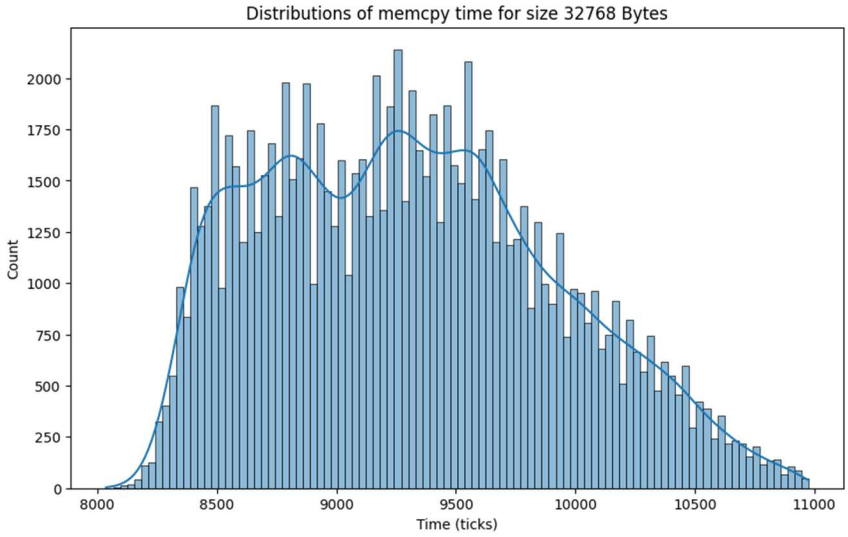

Distributions of memcpy time for size 1024 Bytes
Count
Time (ticks)
0
2000
4000
6000
8000
10000
12000
14000
400
450
500
550
600

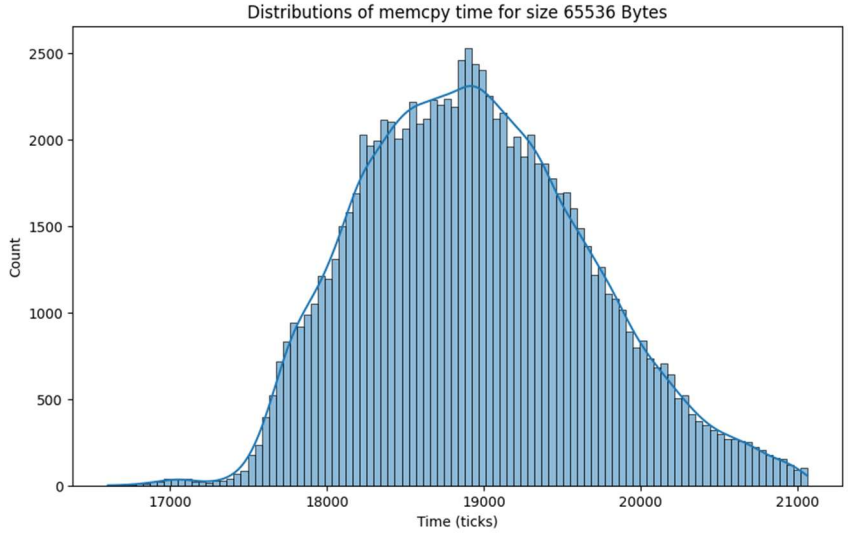

Distributions of memcpy time for size 2048 Bytes
Count
Time (ticks)
0
2500
5000
7500
10000
12500
15000
17500
600
800
1000
1200
1400

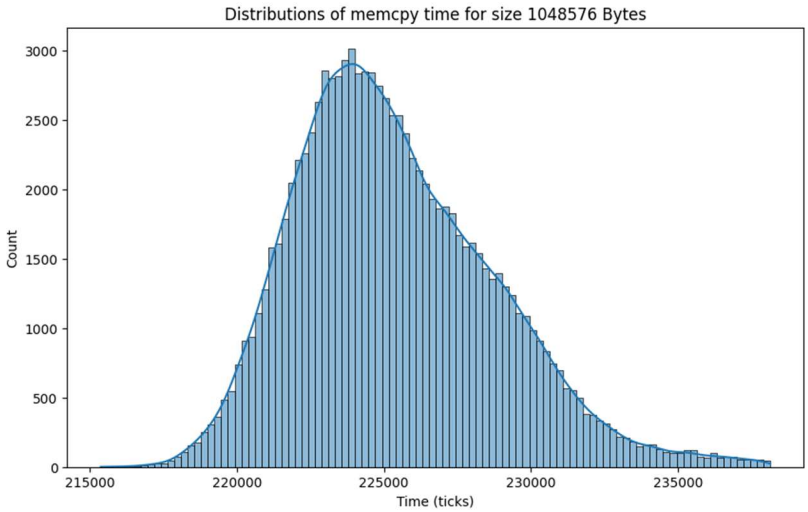

Distributions of memcpy time for size 4096 Bytes
Count
Time (ticks)
0
2000
4000
6000
8000
10000
12000
14000
1400
1600
1800
2000
2200
2400

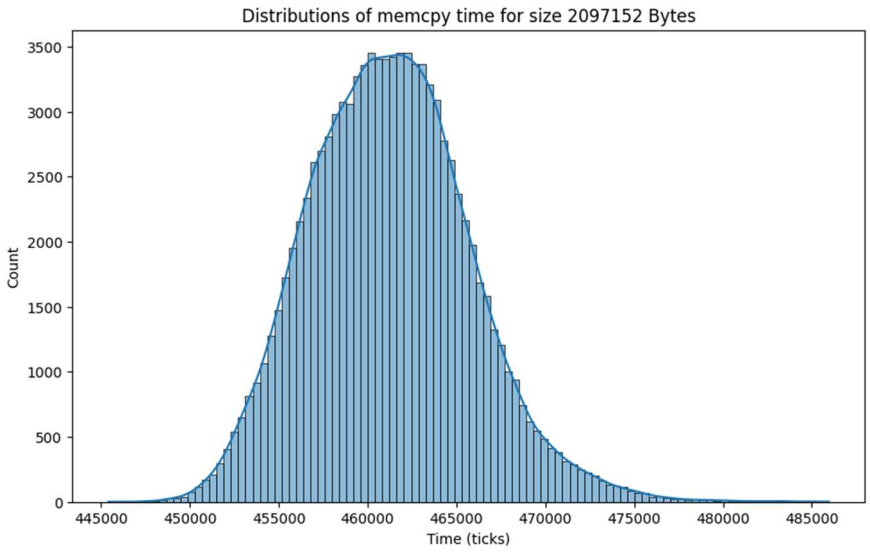

Distributions of memcpy time for size 8192 Bytes
Count
Time (ticks)
0
1000
2000
3000
4000
5000
6000
2500
3000
3500
4000


Distributions of memcpy time for size 16384 Bytes
Count
Time (ticks)
0
1000
2000
3000
4000
5000
4000
4500
5000
5500
6000


Distributions of memcpy time for size 32768 Bytes
Count
Time (ticks)
0
250
500
750
1000
1250
1500
1750
2000
8000
8500
9000
9500
10000
10500
11000


Distributions of memcpy time for size 65536 Bytes
Count
Time (ticks)
0
500
1000
1500
2000
2500
17000
18000
19000
20000
21000


Distributions of memcpy time for size 1048576 Bytes
Count
Time (ticks)
0
500
1000
1500
2000
2500
3000
215000
220000
225000
230000
235000



Code for the question 3

```
#include <stdint.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/mman.h>

#define REPEAT 100
#define BUFFER_SIZE (64 * 1024 * 1024) // 64MB to ensure we're accessing DRAM
#define STRIDE_SIZE (8 * 1024) // 8KB stride, typical DRAM row size

inline void clflush(volatile void *p) {
asm volatile ("clflush (%0)" :: "r"(p));
}

inline uint64_t rdtsc() {
unsigned long a, d;
asm volatile ("rdtsc" : "=a" (a), "=d" (d));
return a | ((uint64_t)d << 32);
}

inline void memory(void *dst, void *src, size_t n) {
memcpy(dst, src, n);
}

long int rep;
```

```
void test_open_vs_closed_row() {
// Allocate large buffers to ensure we're working in DRAM
char* buffer1 = (char*) mmap(NULL, BUFFER_SIZE, PROT_READ | PROT_WRITE,
MAP_PRIVATE | MAP_ANONYMOUS, -1, 0);
char* buffer2 = (char*) mmap(NULL, BUFFER_SIZE, PROT_READ | PROT_WRITE,
MAP_PRIVATE | MAP_ANONYMOUS, -1, 0);
if (buffer1 == MAP_FAILED || buffer2 == MAP_FAILED) {
printf("Memory allocation failed\n");
return;
}
// Initialize buffers
for (int i = 0; i < BUFFER_SIZE; i++) {
buffer1[i] = (char)(i % 256);
buffer2[i] = 0;
}
uint64_t start, end;
uint64_t time_first_access, time_second_access, time_third_access;
volatile char *addr1 = buffer1;
volatile char *addr2 = buffer1 + STRIDE_SIZE; // Different row
// Flush both addresses
clflush(addr1);
clflush(addr2);

// First access to row 1
start = rdtsc();
*addr1 = 'C';
end = rdtsc();
time_first_access = end - start;
printf("First access to row 1: %llu ticks\n", time_first_access);
clflush(addr1);
clflush(addr2);
// Access to different row2
start = rdtsc();
*addr2 = 'D';
end = rdtsc();
time_second_access = end - start;
printf("First access to row2: %llu ticks\n", time_second_access);

clflush(addr1);
clflush(addr2);
// Access to same row2
start = rdtsc();
*addr2 = 'D';
end = rdtsc();
```

```
time_third_access = end - start;
printf("Second access to row2: %llu ticks\n", time_third_access);

// Clean up
munmap(buffer1, BUFFER_SIZE);
munmap(buffer2, BUFFER_SIZE);
}

int main(int ac, char **av) {
printf("Testing DRAM Row Buffer Policy\n");
printf("==============================\n");
test_open_vs_closed_row();
return 0;
}
```